COGNIZANT-DIGITAL-NURTURE-5.0:
Deepskilling-HandsOn- Design Patterns & Principles-

Exercise-1:


```java
package com.cognizant.singleton;


/**
 * Logger class implementing Singleton Pattern.
 * Ensures only one logger instance exists.
 */


public class Logger {
    private static final Logger instance=new Logger();

    private Logger() {
        System.out.println("Logger instance created");
    }

    public static Logger getInstance() {
        return instance;
    }

    public void log(String message) {
        System.out.println("[LOG]"+message);
    }

}

```

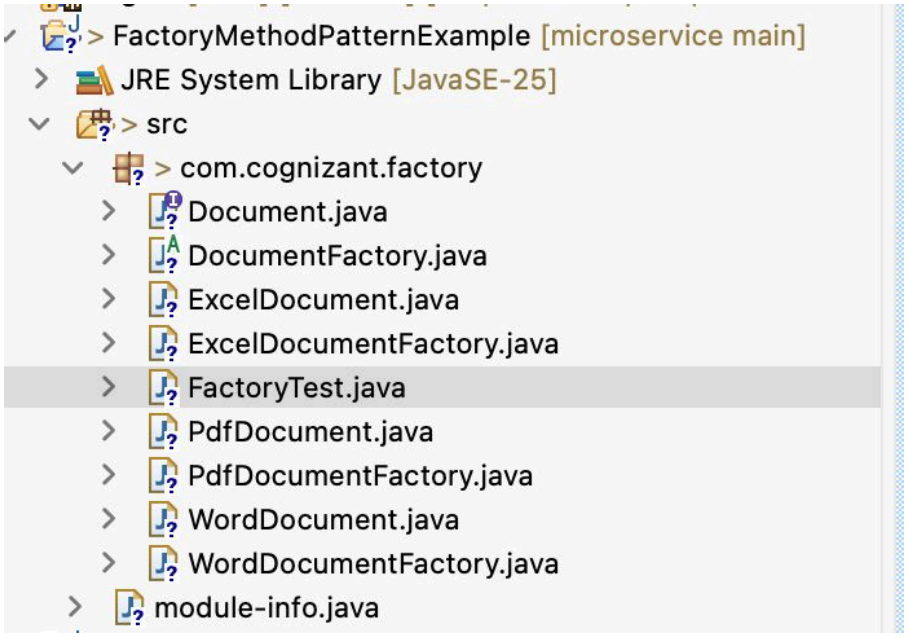
```java
package com.cognizant.singleton;

public class SingletonTest {
    public static void main(String[] args)
    {
        Logger logger1=Logger.getInstance();
        Logger logger2=Logger.getInstance();

        logger1.log("Application started");
        logger2.log("Database Connected");

        System.out.println("Logger1 HashCode : "+logger1.hashCode());
        System.out.println("Logger2 HashCode : "+logger2.hashCode());

        if(logger1 == logger2) {
            System.out.println("Singleton Pattern Verified-Same Instance");
        }

    }
}
```

Problems @ Javadoc Declaration Console

<terminated> SingletonTest [Java Application] /Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin/java (19 Jun 2026, 22:48:32 – 22:48:

```
Logger instance created
[LOG]Application started
[LOG]Database Connected
Logger1 HashCode : 603742814
Logger2 HashCode : 603742814
Singleton Pattern Verified-Same Instance
```

Exercise-2:



Document.java

```java
package com.cognizant.factory;

public interface Document {
    void open();

}

```

Document.java | DocumentFact... | WordDocument... | PdfDocumentF... | Fa

```java
package com.cognizant.factory;

public class PdfDocumentFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new PdfDocument();
    }
}
```

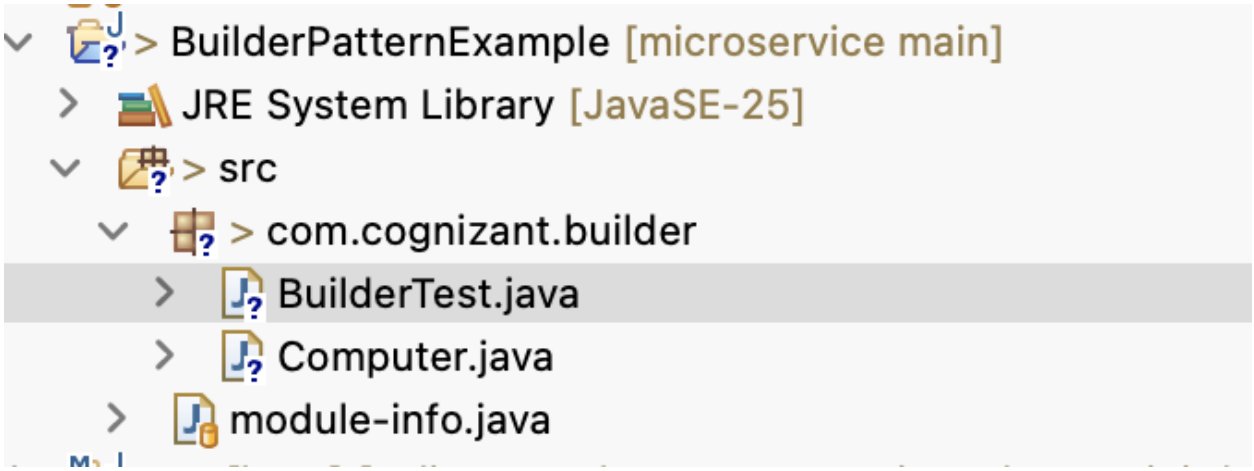
Document.java | WordDocument... | DocumentFact...

```java
package com.cognizant.factory;

public abstract class DocumentFactory {

    public abstract Document createDocument();

}
```

```java
public class FactoryTest {

    public static void main(String[] args) {

        DocumentFactory pdfFactory =
                new PdfDocumentFactory();

        Document pdf =
                pdfFactory.createDocument();

        pdf.open();


        DocumentFactory wordFactory =
                new WordDocumentFactory();

        Document word =
                wordFactory.createDocument();

        word.open();


        DocumentFactory excelFactory =
                new ExcelDocumentFactory();

        Document excel =
                excelFactory.createDocument();

        excel.open();
    }
}
```

```
<terminated> FactoryTest [Java Application] /Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin/java  (20 Jun 2026, 17:01:21 –
PDF Document Opened
Word Document Opened
Excel Document Opened
```

Exercise-3:

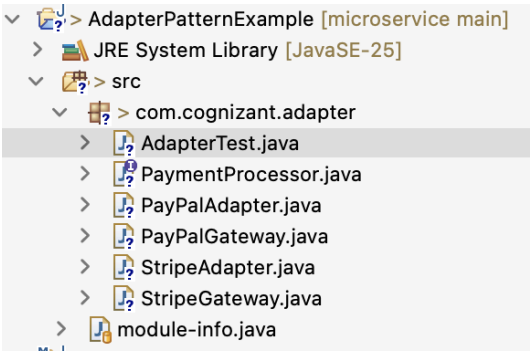



```java
package com.cognizant.builder;

public class Computer {

    private String cpu;
    private String ram;
    private String storage;

    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
    }

    public void display() {
        System.out.println("CPU     : " + cpu);
        System.out.println("RAM     : " + ram);
        System.out.println("Storage : " + storage);
    }

    public static class Builder {

        private String cpu;
        private String ram;
        private String storage;

        public Builder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }

        public Builder setRAM(String ram) {
            this.ram = ram;
            return this;
        }

        public Builder setStorage(String storage) {
            this.storage = storage;
            return this;
        }

        public Computer build() {
            return new Computer(this);
        }
    }
}
```

```java
package com.cognizant.builder;

public class BuilderTest {

    public static void main(String[] args) {

        Computer gamingPC =
                new Computer.Builder()
                        .setCPU("Intel i7")
                        .setRAM("16GB")
                        .setStorage("512GB SSD")
                        .build();

        gamingPC.display();

        System.out.println();

        Computer officePC =
                new Computer.Builder()
                        .setCPU("Intel i5")
                        .setRAM("8GB")
                        .setStorage("256GB SSD")
                        .build();

        officePC.display();
    }
}
```

```
<terminated> BuilderTest [Java App
CPU     : Intel i7
RAM     : 16GB
Storage : 512GB SSD

CPU     : Intel i5
RAM     : 8GB
Storage : 256GB SSD
```

Exercise-4:



```java
package com.cognizant.adapter;

public class AdapterTest {

    public static void main(String[] args) {

        PaymentProcessor paypal =
                new PayPalAdapter(
                        new PayPalGateway());

        paypal.processPayment(1000);

        PaymentProcessor stripe =
                new StripeAdapter(
                        new StripeGateway());

        stripe.processPayment(2000);
    }
}
```

PaymentProce... | StripeGatewa... | PayPalAdapte

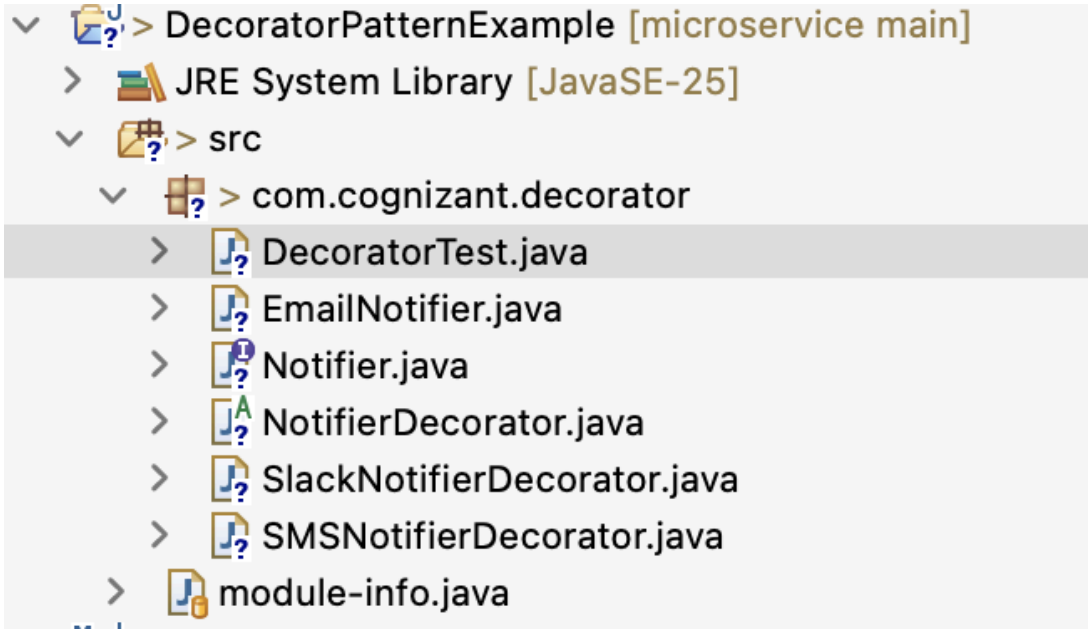
```java
package com.cognizant.adapter;

public interface PaymentProcessor {

     void processPayment(double amount);

}
```

```java
package com.cognizant.adapter;

public class PayPalAdapter
        implements PaymentProcessor {

    private PayPalGateway paypal;

    public PayPalAdapter(PayPalGateway paypal) {
        this.paypal = paypal;
    }

    @Override
    public void processPayment(double amount) {
        paypal.makePayment(amount);
    }
}
```

```java
package com.cognizant.adapter;

public class PayPalGateway {

    public void makePayment(double amount) {
        System.out.println(
                "Payment processed through PayPal: ₹" + amount);
    }
}
```

```
Payment processed through PayPal: ₹1000.0
Payment processed through Stripe: ₹2000.0
```

Exercise-5:




```
Email Notification: Your order has been shipped!
SMS Notification: Your order has been shipped!
Slack Notification: Your order has been shipped!
```

```java
package com.cognizant.decorator;

public class DecoratorTest {

    public static void main(String[] args) {

        Notifier notifier =
                new SlackNotifierDecorator(
                        new SMSNotifierDecorator(
                                new EmailNotifier()));

        notifier.send(
                "Your order has been shipped!");
    }
}
```

```java
package com.cognizant.decorator;

public interface Notifier {

    void send(String message);

}
```

```java
package com.cognizant.decorator;

public abstract class NotifierDecorator
        implements Notifier {

    protected Notifier notifier;

    public NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }

    @Override
    public void send(String message) {
        notifier.send(message);
    }
}
```